SQL FOR DATA ANALYSIS · LESSON 19 OF 20

Incremental Loading and Data-Quality SQL

Load only what has changed, using watermarks, MERGE and upserts, and protect the warehouse with reusable data-quality checks for nulls, duplicates, orphans, ranges and freshness.

Gap-fill lesson | 2026 | Advanced | 150 min

By the end of this lesson you can

- Explain full versus incremental loads and when each is appropriate
- Use a watermark (high-water mark) to pick up only new and changed rows
- Write an upsert with MERGE (or INSERT ... ON CONFLICT) that does not create duplicates
- Run a standard set of data-quality checks and report failures with counts

Datasets used

`superstore_sales.csv`

Why incremental loads matter

A full reload copies every row from the source each night. That is fine for ten thousand rows and painful for ten million. An **incremental load** copies only rows that are new or changed since the last run. It is faster, cheaper, and easier to recover from a failed run, provided it is designed correctly.

CASE STUDY **The nightly sales feed**

The order system writes new orders and corrections all day. The warehouse must show yesterday's sales by 8 a.m. without reloading three years of history every night, and without double-counting the orders that were corrected. The engineer's job is to design the load and to prove the numbers match.

The watermark

A **watermark** is the highest value of a column that the last successful load reached, most often a timestamp such as `updated_at`. The next run asks for rows strictly greater than the watermark.

SQL

```sql
-- Pretend the warehouse table already holds orders up to 30 November 2025
CREATE TABLE fact_orders AS
SELECT row_id, order_id, order_date, ship_date, ship_mode, customer_id, sales, profit
FROM superstore
WHERE order_date ⩽ DATE '2025-11-30';

-- Watermark: the latest order date already loaded
SELECT MAX(order_date) AS watermark FROM fact_orders;

-- The incremental batch: only rows after the watermark
SELECT COUNT(*) AS new_rows, MIN(order_date) AS first_new, MAX(order_date) AS last_new
FROM superstore
WHERE order_date > (SELECT MAX(order_date) FROM fact_orders);
```

RESULT

RESULT SET 1

watermark
2025-11-30

1 rows × 1 columns

RESULT SET 2

new_rows	first_new	last_new
262	2025-12-01	2025-12-31

1 rows × 3 columns

The watermark is only as reliable as the column behind it. Two common traps:

- **Late-arriving rows**: a row dated yesterday may arrive today. A watermark on the event date misses it; a watermark on the arrival or `updated_at` timestamp catches it.
- **Corrected rows**: an updated order keeps its old date, so a date-based watermark never sees the change. Use `updated_at`, or re-load a safety window (for example, the last seven days) and let the upsert handle repeats.

::::tip A safety window Many teams reload the last few days on every run and rely on an idempotent upsert. The extra rows cost little, and the design survives late corrections without special logic. ::::

Upserts: insert new rows and update changed ones

An **upsert** inserts a row when its key is new and updates it when the key exists. Without it, a re-run creates duplicates. Standard SQL uses `MERGE`; PostgreSQL and DuckDB offer `INSERT ... ON CONFLICT`. The example below uses `INSERT ... ON CONFLICT` on the line-level key, which DuckDB supports.

SQL

```sql
-- Target table keyed on the order line
CREATE TABLE fact_lines (
    row_id     BIGINT PRIMARY KEY,
    order_id   VARCHAR,
    product_id VARCHAR,
    sales      DOUBLE,
    quantity   BIGINT,
    discount   DOUBLE,
    profit     DOUBLE
);

INSERT INTO fact_lines
SELECT row_id, order_id, product_id, sales, quantity, discount, profit
FROM superstore WHERE row_id ⩽ 100;

-- Incoming batch: 50 lines, some of them corrected (same row_id, changed profit)
CREATE TEMP TABLE incoming AS
SELECT row_id, order_id, product_id, sales, quantity, discount,
       CASE WHEN row_id ⩽ 10 THEN profit + 100 ELSE profit END AS profit
FROM superstore WHERE row_id BETWEEN 60 AND 110;

-- Upsert: new keys are inserted, existing keys are updated
INSERT INTO fact_lines
SELECT * FROM incoming
ON CONFLICT (row_id) DO UPDATE SET
    sales = excluded.sales,
    quantity = excluded.quantity,
    discount = excluded.discount,
    profit = excluded.profit;

-- Check: no duplicate keys, and the expected total row count
SELECT COUNT(*) AS total_rows, COUNT(DISTINCT row_id) AS distinct_keys FROM fact_lines;
```

RESULT

```
Commands completed successfully.
(100 rows affected)
Commands completed successfully.
(51 rows affected)
```

total_rows	distinct_keys
110	110

1 rows × 2 columns

After the run, `total_rows` equals `distinct_keys`: the rows that overlapped were updated, not copied. Running the same batch twice produces the same result, which is the property called **idempotency**. Idempotent loads can be re-run safely after a failure.

::::warn Plain INSERT is not an incremental load Inserting the batch without a key check doubles the overlapping rows. If a job failed halfway and was restarted, the totals would be wrong and nobody would see it without a check. ::::

Data-quality checks

Good pipelines run checks after each load and stop or alert when a check fails. The five checks below cover most cases. Each query returns the number of failing rows, so zero means pass.

1. Nulls in required columns

SQL

```sql
SELECT COUNT(*) AS failing_rows
FROM superstore
WHERE order_id IS NULL OR order_date IS NULL OR customer_id IS NULL OR product_id IS NULL;
```

RESULT

failing_rows
0

1 rows × 1 columns

2. Duplicate keys

SQL

```sql
SELECT COUNT(*) AS duplicated_keys
FROM (
    SELECT row_id FROM superstore GROUP BY row_id HAVING COUNT(*) > 1
) d;
```

RESULT

duplicated_keys
0

1 rows × 1 columns

3. Orphans: rows whose parent does not exist

Here every order line must have a product in the product table. An orphan means the product was deleted in the source, or the join key was typed incorrectly.

SQL

```sql
SELECT COUNT(*) AS orphan_lines
FROM superstore s
LEFT JOIN products p ON p.product_id = s.product_id
WHERE p.product_id IS NULL;
```

RESULT

orphan_lines
0

1 rows × 1 columns

4. Range checks

Quantities and discounts have business limits. A discount above 100% or a negative quantity is almost always a source error.

SQL

```sql
SELECT
    SUM(CASE WHEN quantity <= 0 THEN 1 ELSE 0 END)          AS non_positive_quantity,
    SUM(CASE WHEN discount < 0 OR discount > 1 THEN 1 ELSE 0 END) AS discount_out_of_range,
    SUM(CASE WHEN sales < 0 THEN 1 ELSE 0 END)               AS negative_sales
FROM superstore;
```

RESULT

non_positive_quantity	discount_out_of_range	negative_sales
0	0	0

1 rows × 3 columns

5. Freshness

A load that succeeds but brings no recent data is a failure too. This check reports how many days old the newest order is.

```sql
SELECT MAX(order_date) AS latest_order,
       DATE_DIFF('day', MAX(order_date), DATE '2025-12-31') AS days_behind_reference
FROM superstore;
```

RESULT

latest_order	days_behind_reference
2025-12-31	0

1 rows × 2 columns

The reference date stands in for "today" so that the example is repeatable. In production, use `CURRENT_DATE`.

A reusable check table

Store results so that trends can be watched over time, not only at the moment of a failure.

```sql
CREATE TABLE IF NOT EXISTS dq_results (
    check_name VARCHAR,
    failing_rows BIGINT,
    checked_on DATE
);

INSERT INTO dq_results
SELECT 'null_required_columns', COUNT(*), DATE '2025-12-31'
FROM superstore WHERE order_id IS NULL OR order_date IS NULL OR customer_id IS NULL OR
product_id IS NULL
UNION ALL
SELECT 'orphan_product_lines', COUNT(*), DATE '2025-12-31'
FROM superstore s LEFT JOIN products p ON p.product_id = s.product_id WHERE p.product_id IS
NULL;

SELECT check_name, failing_rows,
       CASE WHEN failing_rows = 0 THEN 'PASS' ELSE 'FAIL' END AS status
FROM dq_results;
```

RESULT

```
Commands completed successfully.
(2 rows affected)
```

check_name	failing_rows	status
null_required_columns	0	PASS
orphan_product_lines	0	PASS

2 rows × 3 columns

::::key Rules of thumb - Use a watermark on an arrival or update timestamp, and a safety window for late data. - Upsert on a stable key so that re-running a load does not create duplicates. - Check nulls, duplicates, orphans, ranges and freshness after every load, and record the results. - A check that returns a count is easy to alert on; a check that returns rows is easy to investigate. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is a watermark in an incremental load?

A The total row count

B The highest value of a column that the last successful load reached

C The name of the source system

D A password for the warehouse

Answer: B. The next run loads rows beyond the watermark.

2. Why is a date-based watermark risky?

A Dates cannot be compared

B Late-arriving or corrected rows may keep an old date and be missed

C Dates are always duplicated

D It is slower than a full load

Answer: B. Updates to old rows do not move the event date forward.

3. What makes a load idempotent?

A It runs only once

B Running it again with the same input gives the same result

C It deletes the target first

D It uses no keys

Answer: B. Idempotent loads are safe to repeat after a failure.

4. A data-quality check that returns 0 usually means:

A The check failed

B The check passed, because no rows broke the rule

C The table is empty

D The query has a syntax error

Answer: B. Checks in this lesson count failing rows, so zero is a pass.

5. An orphan row is one that:

- **A** Has no date
- **B** Refers to a parent key that does not exist in the parent table
- **C** Is a duplicate
- **D** Has a negative value

Answer: B. Orphans break joins and silently drop rows from reports.

EXERCISE **Freshness and volume alerts**

Write a single query over `superstore` that returns one row per month of 2025 with the number of order lines and the sales total. Add a column `alert` that shows 'LOW VOLUME' when the month's line count is less than 80% of the average monthly line count for 2025, otherwise 'OK'.

Show solution (worked answer)

Hint: use a CTE with `DATE_TRUNC('month', order_date)` grouped by month, then a window `AVG(lines) OVER ()` to get the average, and a `CASE` expression comparing each month to `0.8 * average`.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.